

Лабораторная работа №14: Программирование в оболочке ОС UNIX Семафоры, реализация map и генератор случайных последовательностей

Чжан Вэньцзюнь

РУДН

2026-05-14

Цель работы

- Изучить программирование в оболочке UNIX
- Освоить ветвления и циклы
- Реализовать упрощённый механизм семафоров
- Создать аналог команды `map`
- Научиться генерировать случайные последовательности

Задание 1: Упрощённый механизм семафоров

Постановка задачи

Разработать командный файл:

- ожидающий освобождения ресурса в течение времени $t1$
- использующий ресурс в течение времени $t2$
- реализующий блокировку через `mkdir`

semaphore.sh

```
#!/bin/bash

t1=${1:-10}
t2=${2:-3}
lockfile=${3:-/tmp/my_resource.lock}

echo "Процесс $$ запущен. t1=$t1 t2=$t2 lock=$lockfile"

waited=0

while [ $waited -lt $t1 ]; do
    if mkdir "$lockfile" 2>/dev/null; then
        echo "Процесс $$ получил ресурс (ожидал $waited с"

        sleep $t2

        echo "Процесс $$ освобождает ресурс"
```

Запуск semaphore.sh

Терминал 1

```
tty  
/dev/pts/1
```

```
./semaphore.sh 15 5
```

Вывод:

```
Процесс 1234 запущен. t1=15 t2=5 lock=/tmp/my_resource.1  
Процесс 1234 получил ресурс (ожидал 0 сек.)  
Процесс 1234 освобождает ресурс
```

Терминал 2

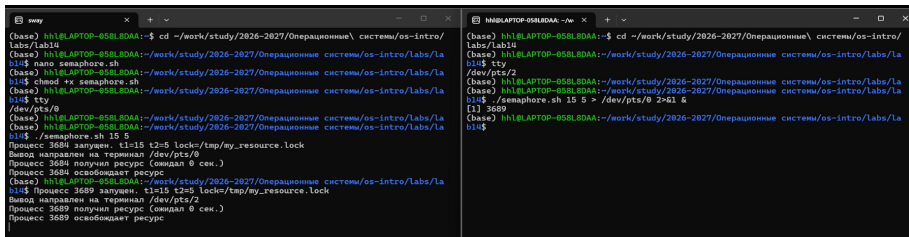
```
tty  
/dev/pts/2  
  
./semaphore.sh 15 5 > /dev/pts/1 2>&1 &
```

Вывод:

```
[1] 1235
```

Результат работы семафора

- процессы корректно ожидают освобождения ресурса
- используется атомарная операция mkdir
- обеспечивается взаимное исключение



```
(base) hhl@LAPTOP-058L8DAA:~$ cd ~/work/study/2026-2027/Операционные\ системы/os-intro/labs/lab14
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ nano semaphore.sh
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ chmod +x semaphore.sh
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ tty
/dev/pts/0
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ ./semaphore.sh 15 5
Процесс 3684 запущен. t1=15 t2=5 lock=/tmp/my_resource.lock
Вывод направлен на терминал /dev/pts/0
Процесс 3684 получил ресурс (ожидал 0 сек.)
Процесс 3684 освобождает ресурс
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$

(hhl@LAPTOP-058L8DAA: ~) x + -
(hhl@LAPTOP-058L8DAA:~$ cd ~/work/study/2026-2027/Операционные\ системы/os-intro/labs/lab14
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ tty
/dev/pts/2
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ ./semaphore.sh 15 5 > /dev/pts/0 2>&1 &
[1] 3689
(base) hhl@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$
```

Задание 2: Реализация команды man

Постановка задачи

Создать упрощённую версию команды man:

- принимать имя команды
- искать справку в `/usr/share/man/man1`
- выводить содержимое через `zless`


```
#!/bin/bash
```

```
if [ $# -ne 1 ]; then  
    echo "Использование: $0 команда"  
    exit 1  
fi
```

```
manpage="/usr/share/man/man1/${1}.1.gz"
```

```
if [ -f "$manpage" ]; then  
    zless "$manpage"
```

```
else
```

```
    echo "Справка по '$1' не найдена в /usr/share/man/man1/  
    exit 1
```

```
fi
```

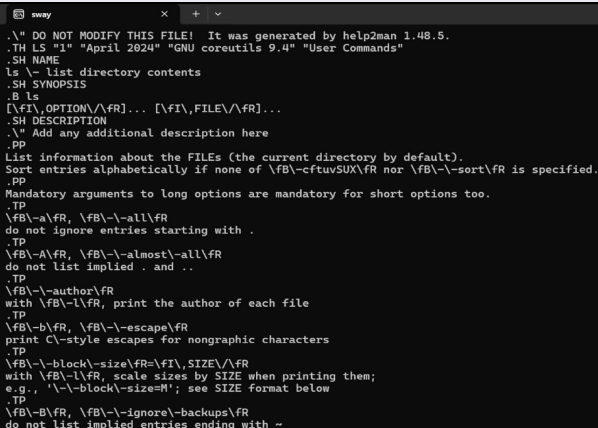
Тестирование myman.sh

Существующая команда

`./myman.sh ls`

Результат:

- отображается справка по команде `ls`



```
sway x + v
.\" DO NOT MODIFY THIS FILE! It was generated by help2man 1.48.5.
.TH LS 1 \" April 2024 \" GNU coreutils 9.4 \" User Commands
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[\fI\,OPTION\[/fR]... [\fI\,FILE\[/fR]...
.SH DESCRIPTION
.\" Add any additional description here
.PP
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of \fB\--ctuvSUX\[/fR nor \fB\--sort\[/fR is specified.
.PP
Mandatory arguments to long options are mandatory for short options too.
.TP
\fB\--a\[/fR, \fB\--all\[/fR
do not ignore entries starting with .
.TP
\fB\--A\[/fR, \fB\--almost\--all\[/fR
do not list implied . and ..
.TP
\fB\--author\[/fR
with \fB\--l\[/fR, print the author of each file
.TP
\fB\--b\[/fR, \fB\--escape\[/fR
print C\--style escapes for nongraphic characters
.TP
\fB\--block\--size\[/fR=\fI\,SIZE\[/fR
with \fB\--l\[/fR, scale sizes by SIZE when printing them;
e.g., '\--block\--size=M'; see SIZE format below
.TP
\fB\--B\[/fR, \fB\--ignore\--backups\[/fR
do not list implied entries ending with ~
```

Несуществующая команда

```
./myman.sh nonexistent_command
```

Вывод:

```
Справка по 'nonexistent_command' не найдена в /usr/share/
```

```
(base) hnt@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ ./myman.sh notexist
Справка по 'notexist' не найдена в /usr/share/man/man1
```

Задание 3: Генератор случайных последовательностей

Постановка задачи

Создать командный файл:

- генерирующий случайные буквы
- использующий \$RANDOM
- поддерживающий длину строки через аргумент

random_letters.sh

```
#!/bin/bash

length=${1:-20}

letters="abcdefghijklmnopqrstuvwxyz"
result=""

for ((i=0; i<length; i++)); do
    idx=$((RANDOM % 26))
    result="${result}${letters:idx:1}"
done

echo "Случайная последовательность: $result"
```

Тестирование random_letters.sh

Стандартный запуск

```
./random_letters.sh
```

Вывод:

Случайная последовательность: kxmzowpqrbcnthgejdsa

Генерация 30 символов

```
./random_letters.sh 30
```

Вывод:

Случайная последовательность: plmoknijbuhvygctfxrdzeswaq1

```
(base) hh1@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ ./random_letters.sh 30
Случайная последовательность: tarubztruxnfyrbnudvdihaxmhunl
(base) hh1@LAPTOP-058L8DAA:~/work/study/2026-2027/Операционные системы/os-intro/labs/lab14$ ./random_letters.sh
Случайная последовательность: sjrmreqnfgzuawhnpzgp
```

1. Ошибка в while

Неверно:

```
1 while [$1 != "exit"]
```

Верно:

```
while [ "$1" != "exit" ]
```


2. Конкатенация строк

```
str="Hello"  
str2="World"  
  
result="$str $str2"  
result="${str}${str2}"
```

3. Аналоги seq

- `echo {1..10}`
- цикл `for`
- цикл `while`

Пример:

```
for ((i=1; i<=10; i++)); do  
    echo $i  
done
```

4. Результат вычисления

```
$((10 / 3))
```

Результат:

```
3
```

5. Отличия zsh от bash

- улучшенное автодополнение
- расширенные glob-шаблоны
- массивы с индексацией от 1
- более гибкая настройка интерфейса

6. Ошибка в for

Неверно:

```
for ((a=1; a <= LIMIT; a++)
```

Верно:

```
for ((a=1; a <= LIMIT; a++)); do  
    команды  
done
```

7. Bash: преимущества и недостатки

Преимущества	Недостатки
Автоматизация задач	Медленная работа
Не требует компиляции	Сложный синтаксис
Есть в любой Linux-системе	Слабые структуры данных
Удобная работа с ОС	Ограниченная отладка

В ходе работы были изучены:

- циклы и условия
- bash-скрипты
- работа с \$RANDOM
- перенаправление ввода-вывода
- реализация семафоров через mkdir
- создание аналога команды man

Полученные навыки позволяют создавать более сложные сценарии автоматизации UNIX.

Раздел 1

Спасибо за внимание!